

Table des matières

STRUCTURES DE DONNÉES

Les Listes	4
Opérations sur une liste	5
Implémentation	7
Insertion classée dans une liste	8
Version itérative & récursive	8
Suppression classée dans une liste	9
Version itérative & récursive	9
Implémentation chaînée	10
Rappel	11
Déclaration d'une liste simplement chaînée	13
En Algorithmique	13
Accès aux champs d'une structure	13
Déclaration d'une liste chaînée en Langage C	14
Accès aux membres d'une structure non pointée	15
Accès aux membres d'une structure pointée	16
Manipulations des listes simplement chaînées	18
Définition itérative d'une liste linéaire chaînée	20
Exemple	20
Définition récursive d'une liste chaînée	20
Primitives de base	21
Quelques Opérations de base	22
Opérations init , estVide & estPlein	22
Fonction de création d'une liste vide	22
Fonction estVide : Liste → Booléenne	22

Fonction estPlein : Liste → Booléenne	22
Parcours d'une liste chaînée	23
Schéma itératif	23
Schéma récursif	24
Application : Calcul de la longueur d'une liste = nombre d'éléments= nombre de cellules	27
Schéma itératif	27
Schéma récursif	28
Accès dans une liste	29
Accès par position	29
Accès par position	30
Schéma récursif	31
Accès associatif = Recherche d'un élément donné	33
Schéma itératif	33
Schéma récursif	34
Recherche d'un élément dans une liste triée	35
Définition d'une liste triée	35
Schéma itératif	35
Schéma récursif	35
Algorithmes d'insertion dans une liste	37
Insertion d'un élément en tête de liste	37
Insertion d'un élément en fin de liste	40
Schéma itératif	40
Version récursive de l'insertion en Queue	43
Algorithme de la fonction dernier	44
Version itératif	44
Schéma itératif : deuxième version	45

Schémas récurrents	46
Insertion dans une liste triée	47
Schéma récursif	47
Schéma itératif	48
Suppression d'un élément dans une liste	49
Suppression du premier élément	49
Suppression de la liste	50
Suppression d'un élément dans une liste	55
Listes avec deux adresses	57
Déclaration la structure en C	57
Primitives de base	58
Insertion en tête	61
Insertion en queue	63
Suppression en tête	64

STRUCTURES DE DONNÉES

- Permettent de gérer, d'organiser et de stocker des données.
- Sont définies à partir d'un ensemble d'opérations qu'elles peuvent effectuer sur les données.
- Une structure de données ne regroupe pas nécessairement des objets du même type.
- Une structure de données à une **interface** qui consiste en un ensemble d'opérations pour insérer, supprimer, accéder, réorganiser, etc. les données.
- Un type de données abstrait (TDA) = définition des propriétés de la structure et de son interface ("cahier des charges")
- On ne s'intéresse pas aux éléments de la structure en question mais aux opérations sur cette structure, indépendamment de la nature des données.

LES LISTES

Une liste est une structure de données qui permet de stocker, en mémoire centrale, un ensemble fini des données de même nature selon un certain ordre.

L'accès aux éléments d'une liste peut se faire de deux façons différentes : accès séquentiel ou directe.

Opérations sur une liste

creatListe : [] $\rightarrow$ Liste

estVideListe : Liste $\rightarrow$ Booléen

estPleinListe : Liste $\rightarrow$ Booléen

Insertions :

Insertion en tête : TElement x Liste $\rightarrow$ Liste

Insertion en queue : TElement x Liste $\rightarrow$ Liste

Insertion d'un élément : TElement x Liste $\rightarrow$ Liste

Insertion Triée d'un élément : TElement x Liste $\rightarrow$ Liste

Insertion par Position : Entier x Liste x TElement $\rightarrow$ Liste

...

Suppressions :

Suppression En tête : Liste $\rightarrow$ Liste

Suppression en Queue : Liste $\rightarrow$ Liste

Suppression d'un élément : TElement x Liste $\rightarrow$ Liste

Suppression Triée d'un élément : TElement x Liste $\rightarrow$ Liste

Suppression par Position : Entier x Liste x TElement $\rightarrow$ Liste

..

Parcours

parcours : Liste $\rightarrow$ []

rechPos : ListexEntier $\rightarrow$ Bool

rechPos : ListexEntier $\rightarrow$ Liste

rechElt : ListexTElement $\rightarrow$ Bool

rechElt : ListexTElement $\rightarrow$ Liste

derniereCellule : Liste $\rightarrow$ Liste

inverse : Liste $\rightarrow$ Liste

...

Autres opérations :

fusion : Liste $\times$ Liste $\rightarrow$ Liste

intersection : Liste $\times$ Liste $\rightarrow$ Liste

Etc

Implémentation

Deux types d'implémentations du type liste sont possibles :

Implémentation contiguë, où la taille de la liste est définie et l'accès aux éléments est directe et,

L'implémentation chaînée où l'accès est séquentiel.

Implémentation contiguë

Deux solutions sont possibles :

Version 1 : statique	Version 2: dynamique
<pre>#define NBELTS MAX 100 typedef ... TElement ; typedef struct{ int nbElts ; TElement tab[NBELTS MAX] ; }Liste ;</pre>	<pre>typedef ... TElement ; typedef struct{ int nbEltsMax ; int nbElts ; TElement *tab; }Liste ;</pre>

- Dans un tableau l'accès aux éléments du tableau se fait d'une **manière directe**.
- Par contre, dans un tableau, l'insertion (resp. Suppression) n'est possible que par un **décalage** de tous les éléments du tableau.
- L'utilisation d'un tableau **est simple**.

Insertion classée dans une liste

Précondition : non est plein liste

Fonction inserElt : TElement x Liste → Liste

Version itérative & récursive

Voir TD

Suppression classée dans une liste

Précondition : non est vide liste

Fonction `suppElt : TElement x Liste → Liste`

Version itérative & récursive

Voir TD

IMPLÉMENTATION CHAÎNÉE

Appelée souvent une liste chaînée.

Les listes chaînées se présentent comme une alternative aux tableaux pour le stockage de certaines données.

Les tableaux présentent quelques inconvénients majeurs :

- Difficilement extensibles : Lorsqu'on crée un tableau, on doit connaître sa taille à l'avance, que ce soit avec l'utilisation ou non de `malloc` (ou `calloc`).

Ainsi, on se retrouve limités par la taille du tableau. Il est cependant possible à tout moment d'agrandir un tableau, mais ceci n'est pas la meilleure solution envisageable.

- Insertion (resp. Suppression) classée n'est possible que par un décalage de tous les éléments.

Les **listes chaînées** permettent de pallier ces inconvénients et offre des réponses satisfaisantes à ces problèmes :

La taille de liste peut être changée durant l'exécution du programme, sa taille dépend de la taille de la mémoire centrale disponible.

L'allocation et la libération de la mémoire s'effectue en fonction des besoins. En crée la mémoire en cas de besoin et on libère si non.

L'accès aux éléments est **séquentiel (est donc non direct)**.

La suppression et l'ajout peuvent se faire à n'importe quel endroit de la liste.

Rappel

Les éléments de la liste ne sont pas stockés de façon contigüe, ni ordonné.

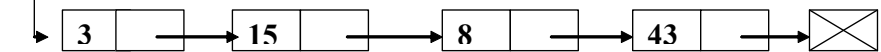
Ainsi le premier élément de la liste peut se trouver à l'adresse 1024, le second à l'adresse 256, le troisième en 532, le quatrième en 2084 etc.

Afin de retrouver la bonne séquence des éléments en mémoire centrale, il est donc nécessaire d'ajouter de l'information à chaque élément de la liste. Cette information permettra de retrouver l'élément suivant en indiquant son **adresse** (= **pointeur**).

Les différents types de listes chaînées

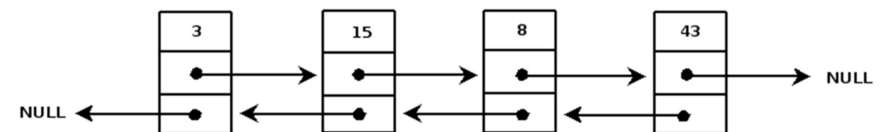
Les listes simplement chaînées : chaque élément dispose d'un pointeur vers l'élément suivant de la liste.

Liste



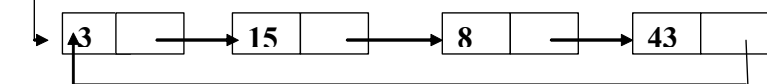
Les listes doublement chaînées : chaque élément dispose de deux pointeurs pointant respectivement vers l'élément précédent et l'élément suivant.

Liste doublement chaînée de 4 valeurs



Les listes circulaires : le dernier élément pointe sur le premier élément de la liste.

Liste



Autres variantes de listes chaînées

- Implémentation avec **deux adresses** (tête et queue)
- Listes avec **sentinelles**

Remarques

- Ces différents types peuvent être mixés selon les besoins.
- Les éléments d'un tableau sont **contigus** en mémoire, les éléments d'une liste chaînée sont **dispersés** en mémoire et reliés entre eux via une série de pointeurs.

Déclaration d'une liste simplement chaînée

En Algorithmique

Type Cellule = Enregistrement (Structure)

Début

donnee : TElement

suivant : Adresse d'une cellule

Fin

Type *Liste = Adresse d'une cellule*

Une adresse s'appelle aussi un **pointeur**.

Accès aux champs d'une structure

si s est une structure (enregistrement) composées de plusieurs champs nommés a,b etc. Alors s.a est la variable correspondant au membre a de la structure.

si p est un pointeur sur un enregistrement à plusieurs champs nommés a,b etc. Alors p.a est la variable correspondant au champ a.

Un pointeur qui ne pointe sur rien a la valeur nul.

Déclaration d'une liste chaînée en Langage C

VERSION 1	VERSION 2
<pre>struct cellule{ TElement donnee; struct cellule *suivant; }; typedef struct cellule *Liste;</pre>	<pre>typedef struct cellule{ TElement donnee; struct cellule *suivant; }Cellule; typedef Cellule *Liste;</pre>

VERSION 3	VERSION 4
<pre>typedef struct cellule{ TElement donnee; struct cellule *suivant; }* Liste ;</pre>	<pre>typedef struct cellule* Liste ; struct cellule{ TElement donnee; struct cellule *suivant; };</pre>

Accès aux membres d'une structure non pointée

On utilise l'opérateur de sélection (.) pour sélectionner chaque membre de la structure.

Exemple

```
struct cellule cel          //↔ Cellule cel ;
```

// si cel est une variable de type **Cellule** alors

cel.donnee désigne le membre donnée de la structure cellule

cel.suivant désigne le membre suivant de la structure cellule

Accès aux membres d'une structure pointée

En langage C, on utilise l'opérateur → pour sélectionner les différents membres d'une structure pointée.

Exemple

```
struct cellule * l;      //Cellule *l;      //Liste l ;
```

Si l est une variable de type liste, alors

l→donne (ou (*l).donnee) désigne le membre donnée de la structure Liste

l→suivant (ou (*l).suivant) désigne le membre Suivant de la structure Liste.

Rappel : Valeur d'adresse NULL

La valeur NULL ne correspond à aucune adresse possible de l'objet.

Affectation :

$L \leftarrow \text{NULL}$ (L=NULL)

NULL :

- permet l'initialisation d'une structure dynamique.
- permet en particulier de tester les limites d'une structure dynamique,

Le contenu d'une variable de type pointeur n'est défini que s'il est différent de NULL.

Remarques

- Dans une liste, il y a autant de cellules que d'éléments.
- Le membre suivant d'une cellule contient l'adresse de la cellule suivante.
- Le membre suivant de la dernière cellule contient la valeur NULL.
- Il suffit de connaître l'adresse de la première cellule pour avoir accès à tous les éléments de la liste.

Une liste chaînée est définie par l'adresse de sa première cellule.

Manipulations des listes simplement chaînées

Création des fonctions afin de réaliser plusieurs opérations sur ces listes.

Opérations sur une liste

creatListeVide : [] → Liste

estVideListe : Liste → Booléen

estPleinListe : Liste → Booléen

Insérer : **TElement x Liste → Liste**

Insertion en tête

Insertion en queue

Insertion d'un élément

Insertion Triée d'un élément

Insertion par Position : Entier x Liste x TElement → Liste

...

Supprimer : **Liste → Liste**

Suppression En tête

Suppression en Queue

Suppression d'un élément : TElement x Liste → Liste

Suppression Triée d'un élément : TElement x Liste → Liste

Suppression par Position : Entier x Liste → Liste

...

Parcours

rechElt : TElement x Liste $\rightarrow$ Bool

rechElt : TElement x Liste $\rightarrow$ Liste

derniereCellule : Liste $\rightarrow$ Liste

Longueur : Liste $\rightarrow$ Entier

Inverser : Liste $\rightarrow$ Liste

...

Autres Opérations

Concaténation : Liste $\times$ Liste $\rightarrow$ Liste

Intersection : Liste $\times$ Liste $\rightarrow$ Liste

Etc

Définition itérative d'une liste linéaire chaînée

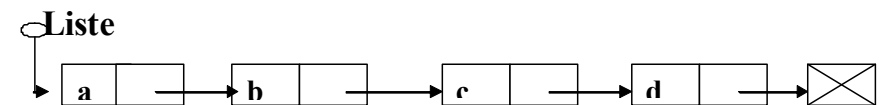
Une liste chaînée est constituée d'un ensemble de cellules (maillons) chaînées entre elles.

L'adresse de la première cellule est l'adresse de la liste.

Autrement dit, une liste chaînée est définie par l'adresse de sa première cellule.

Exemple

La liste linéaire [a, b, c, d] est représentée par la liste chaînée suivante :



Définition récursive d'une liste chaînée

Une liste chaînée est :

- Soit une liste vide
- Soit composée d'une cellule (la première) chaînée à une sous liste (une sous liste est une liste obtenue après suppression de la première cellule).

Remarque

Dans une liste chaînée, chaque élément est lié à son successeur et il n'est donc pas possible d'accéder directement à un élément quelconque de la liste.

Primitives de base

Donnée : retourne la donnée d'une liste

donnee : Liste → TElement

Précondition : non vide liste

Fonction donnee(l) → TElement P.F l : Liste (E) Début renvoyer l.donnee Fin	TElement donnee(Liste l){ return l->donnee ; }
---	--

Suivant : retourne la cellule suivante

suivant : Liste → Liste

Précondition : non vide liste

Fonction suivant(l) → Liste P.F l : Liste Début renvoyer l.suivant Fin	Liste suivant(Liste l){ return l->suivant ; }
--	---

Quelques Opérations de base

Opérations init, estVide & estPlein

Fonction de création d'une liste vide

init : [] → Liste

Fonction init() → Liste Debut Renvoyer NULL Fin	Liste init(void){ return NULL ; }
---	---

Fonction estVide : Liste → Booléenne

Pour savoir si une liste est vide, il suffit de tester si sa première cellule vaut NULL.

Fonction estVide : Liste → Booléenne

Fonction estVideListe(l) → Booléenne P.F l : Liste (E) Début Renvoyer l=NULL Fin	int estVideListe(Liste l){ return l=NULL ; }
--	---

Fonction estPlein : Liste → Booléenne

On suppose qu'une liste n'est jamais plein

Fonction estPlein : Liste → Booléenne

Fonction estPlein(l) → Booléenne P.F l : Liste (E) Début renvoyer faux Fin	int estPlein(Liste l){ return 0; }
--	---

Parcours d'une liste chaînée

parcours : Liste--> traiter les Donnée(s) (ou TElement)

Schéma itératif

Pour l'écriture des algorithmes sous forme itératif, on applique la

définition suivante d'une liste chaînée :

Une liste chaînée est constituée d'un ensemble de cellules (maillons) chaînées entre elles. L'adresse de la première cellule est l'adresse de la liste.

procédure Parcours (l)

P.F l : Liste (E)

Début

Tantque non estVide(l) faire

 traiter(donnee(l))

 l ← suivant(l)

fintq

Fin

Schéma récursif

Pour l'écriture des algorithmes sous forme récursive, on applique la

définition suivante d'une liste chaînée :

Une liste chaînée est :

- Soit une liste vide
- Soit composée d'une cellule (la première) chaînée à une sous liste (obtenue après suppression de la première cellule) et la sous liste est aussi une liste chaînée.

Premier parcours

- si la liste est vide
 - le parcours est terminé
- la liste n'est pas vide
 - Traiter la première cellule
 - Effectuer le parcours de la sous liste

procédure parcours(l)

P.F I : Liste (E)

Début

Si non estVide(l) alors

traiter (donnee(l))

parcours(suivant(l))

Finsi

Fin

Remarques

La récursivité est terminale

Ce parcours traite les cellules de la liste de gauche à droite.

Deuxième parcours

- Si la liste est vide
 - Le parcours est terminé
- La liste n'est pas vide
 - Effectuer le parcours de la sous liste
 - Traiter la première cellule

procédure parcours(l)

P.F I : Liste (E)

Début

Si non estVide(l) alors

parcours(suivant(l))

traiter (donnee(l))

Finsi

Fin

Remarques

La récursivité est non terminale

Ce parcours traite les cellules de la liste de droite à gauche.

Application : Calcul de la longueur d'une liste = nombre d'éléments= nombre de cellules

Fonction longueur : Liste-->Entier

Schéma itératif

- La liste est vide
 - L'algorithme est terminé : la liste contient 0 éléments : $lg \leftarrow 0$
- la liste n'est pas vide
 - La liste à au moins un élément
 - $lg \leftarrow lg + 1$ et $l \leftarrow l.suivant$
 - Itération : tq non estVide(l) faire

Fonction longListe(l) → Entier

P.F I : Liste (E)

Début

$lg \leftarrow 0$

Tantque non estVide(l) faire

$lg \leftarrow lg + 1$

$l \leftarrow suivant(l)$

Fintq

Renvoyer lg

Fin

Schéma récursif

- Si la liste est vide
 - Arrêt de l'algorithme : renvoyer 0
- Si la liste n'est pas vide
 - Elle est donc composée d'une cellule (le premier élément) chaînée à une sous liste (suivant(l)). La longueur de la liste est donc égale à 1 plus la longueur de la sous liste
 - on exécute l'action : renvoyer 1 + longListe(suivant(l))

Fonction longListe(l) → Entier

P.F I : Liste (E)

Début

si estVide(l) alors

renvoyer 0

sinon

renvoyer 1 + longListe(suivant(l))

finsi

Fin

Accès dans une liste

On distingue deux sortes d'accès : accès par position et l'accès associatif.

Accès par position

Accès à la $k^{\text{ième}}$ position dans une liste chaînée quelconque.

accesPosition : Entier X Liste--> Booléenne (ou TElement ou Adresse)

fonction accesPosition(pos, l) → booléenne

P.F : pos : Entier (E)

l : Liste (E)

Début

cpt ← 1

tanque (non estVide(l) && cpt < pos) faire

cpt ← cpt + 1

l ← suivant(l)

Fintq

Renvoyer non (estVide(l))

Fin

Accès par position

Cette fonction retourne l'adresse du $k^{\text{ième}}$ élément s'il existe et retourne NULL sinon.

accesPosition : Entier X Liste--> Liste

fonction accesPosition(pos, l) → Liste

Debut

cpt ← 1

Tanque (non estVide(l) et cpt < pos) faire

cpt ← cpt + 1

l ← suivant(l)

Fintq

Renvoyer l

Fin

Schéma récursif

- si la liste est vide
 - Arrêt de l'algorithme : renvoyer Faux
- si la liste n'est pas vide
 - si pos=1
 - Arrêt de l'algorithme : renvoyer Vrai
 - sinon : la liste est non vide et pos > 1
 - On exécute l'action : renvoyer (pos-1, l.suivant)

fonction accesPosition(pos, l) → Booléenne

Debut

Si (estVide(l)) alors

Renvoyer Faux

Sinon si pos=1 alors

Renvoyer Vrai

Sinon

renvoyer accesPosition(pos-1, suivant(l))

Finsi

Finsi

Fin

fonction accesPosition(pos, l) → Liste

Debut

Si estVide(l) ou (pos=1) alors

renvoyer l

Sinon

renvoyer accesPosition(pos-1, suivant(l))

Finsi

Fin

Accès associatif = Recherche d'un élément donné

rechercheElement : TElement X Liste --> Booléenne (ou Liste)

Schéma itératif

Fonction rechercheElement(elt, l) → Booléenne

P.F elt : TElement (E)

l : Liste (E)

Début

Tant que (non estVide(l) et donnee(l) ≠ elt) faire

l ← suivant(l)

Fintq

Renvoyer non estVide(l)

Fin

Fonction RechercheElement(elt, l) → Liste

Début

Tant que (non estVide(l)) et (donne(l) ≠ elt) faire

l ← suivant(l)

Fintq

Renvoyer l

Fin

Schéma récursif

Fonction RechercheElement(elt, l) → Booléenne

Début

Si estVide(l) alors

Renvoyer Faux

Sinon si donnee(l)=elt alors

Renvoyer Vrai

Sinon

Renvoyer RechercheElement(elt, suivant(l))

Finsi

Finsi

Fin

Fonction RechercheElement(elt, l) → Liste

Début

Si estVide(l) ou donnee(l)=elt alors

Renvoyer l

Sinon

Renvoyer RechercheElement(elt, suivant(l))

Finsi

Fin

Recherche d'un élément dans une liste triée

Définition d'une liste triée

- Une liste vide est triée
- Une liste composée d'un seul élément est triée
- Une liste de plus d'un élément est triée si tous les éléments consécutifs vérifient la relation d'ordre ($\leq$) suivante :
Non estVide(l) et non estVide(suivant(l)) et
 $donnee(l) \leq donnee(suivant(l))$

Schéma itératif

Voir TD

Schéma récursif

Fonction RechercheElementListeTrie(e, l) → Booléenne

Début

Si estVide(l) ou $donnee(l) > e$ alors

Renvoyer Faux

Sinon si $donnee(l) = e$ alors

Renvoyer Vrai

sinon

Renvoyer **RechercheElementListe(e, suivant(l))**

Finsi

Finsi

Fin

Fonction RechercheElementListeTrie(e, l) → Liste

Début

Si estVide(l) ou $donnee(l) > e$ alors

Renvoyer NULL

Sinon

Si $donnee(l) = e$ alors

Renvoyer l

Sinon

Renvoyer **RechercheElementListe(e, suivant(l))**

Finsi

Fin

Algorithmes d'insertion dans une liste

Insertion d'un élément en tête de liste

C'est un algorithme très important que nous utiliserons très souvent pour réaliser les autres insertions.

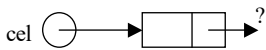
Cette insertion est particulière puisqu'elle modifie l'adresse de la liste.

inserTête : TElement X Liste --> Liste

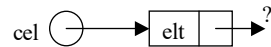
Principe

❖ Création d'une nouvelle cellule

$cel \leftarrow \text{CreationNouvelleCellule}()$:



❖ Remplissage du membre donné de la cellule

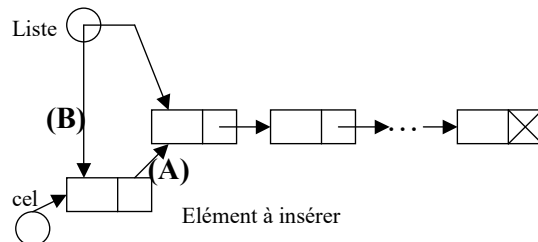


$\text{donnee}(Cel) \leftarrow elt$

❖ Réalisation des chaînages (A) et (B) dans cet ordre :

$cel.suivant \leftarrow l$ /* Chaînage A */

$l \leftarrow cel$ /* Chaînage B */



Remarque :

Les éléments suivants sont décalés automatiquement d'une position.

La fonction insertion est alors

Procédure inserTête(elt, l)

P.F elt : TElement (E)

l : liste (E/S)

Début

$cel \leftarrow \text{CreationNouvelleCellule}()$

$cel.donnee \leftarrow elt$

$cel.suivant \leftarrow l$ /* Liaison A */

$l \leftarrow cel$ /* Liaison B */

Fin

void inserTête(TElement elt, Liste *l){

Liste cel ;

$cel = (Liste) \text{ malloc}(\text{sizeof}(\text{struct cellule})) ;$

$cel \rightarrow donnee = elt ;$

$cel \rightarrow suivant = *l ;$

$*l = cel ;$

}

Liste insertTête(TElement elt, Liste l){

Liste cel ;

 Cel=(Liste) malloc(sizeof(struct cellule)) ;

 Cel→donnee=elt ;

 Cel→Suivant=l ;

 l=cel ;

 return l ;

}

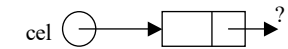
Insertion d'un élément en fin de liste

inserQueue : TElement X Liste--> Liste

Schéma itératif

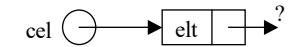
- si la liste est vide, on est ramené à une insertion en tête de liste.
- On procède comme suit si la liste n'est pas vide :
- ❖ Création d'une nouvelle cellule

$Cel \leftarrow \text{CreationNouvelleCellule}()$:

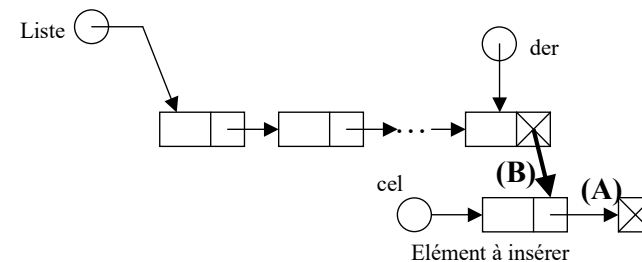


- ❖ Remplissage du champ information de la cellule

$\text{Info}(Cel) \leftarrow \text{elt}$



- ❖ Réalisation des chaînages (A) et (B).



Il faut connaître l'adresse de la dernière cellule « der » .

On suppose que dernier est une fonction qui renvoie la dernière cellule d'une liste.

La fonction insertion en fin est alors

procédure inserQueue(elt, l)

P.F Elt : TElement(E) l : liste (E/S)

Début

Si estVide(l) alors
 InserTete(elt, l)

Sinon

 der ← dernier(l)

(1) cel ← CreationNouvelleCellule()

(2) Cel.donnee ← elt

(3) cel.Suivant ← NULL (ou suivant(der)) /* Liaison A */

(4) der.Suivant ← cel /* Liaison B */

Finsi

Fin

Remarque : Les actions (1), (2), (3) et (4) correspondent à l'appel suivant : inserTete(elt, der.Suivant)

Procédure InserQueue(elt, l)

Début

Si estVide(l) alors
 inserTete(elt, l)

Sinon

 der ← dernier(l)

 InserTete(elt, suivant(der))

Finsi **// Attention traduction en C**

Fin

Version récursive de l'insertion en Queue

Raisonnement

- si la liste est vide
→ on insère l'élément en tête : `inserTete(elt, l)`
- si la liste n'est pas vide
→ On insère l'élément en fin de la sous liste de l :
`inserQueue(elt, suivant(l))`

procédure `inserQueue(elt, l)`

P.F elt : TElement (E) l : Liste (E/S)

Début

si `estVide(l)` alors

`inserTete(elt, l)`

sinon

`inserQueue(elt, suivant(l))`

Finsi **// Attention traduction en C**

Fin

Algorithme de la fonction dernier

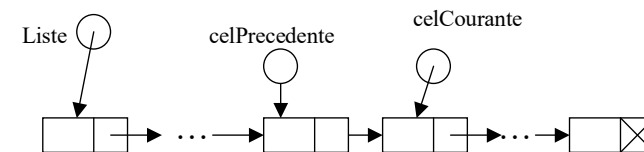
dernier : Liste --> Liste

Hypothèse

Pré-condition : On suppose que la liste n'est pas vide

Version itératif

On effectue un parcours de tous les éléments de la liste en préservant à chaque fois l'adresse de la cellule précédente.



Fonction `dernier(l) → Liste (=Adresse de la dernière cellule)`

P.F L : Liste (E)

Début

`celCourante ← l`

 Tantque non `estVide(celCourante)` faire

`CelPrecedente ← celCourante`

`CelCourante ← Suivant(celCourante)`

 Fintq

 Renvoyer `CelPrecedente`

Fin

Variables Intermédiaires

`CelCourante`, `CelPrecedente` : Liste

Schéma itératif : deuxième version

Cette deuxième version consiste à ne pas utiliser des variables auxiliaire précédent mais à noter que : l et suivant(l) contiennent les adresses de deux cellules consécutifs.

Il suffit de parcourir la liste en s'arrêtant quand la liste est réduite à une seule cellule qui est la dernière c'est à dire suivant(l) est vide.

Rappel : Pré-condition : on suppose que la liste est non vide

Fonction dernier(l) → Liste

P.F l : liste (E)

Début

der ← l

Tantque non estVide(suivant(der)) faire

der ← suivant(der)

Fintanque

Renvoyer der

Fin

Schémas récursifs

Hypothèse : on suppose que la liste n'est pas vide

Raisonnement

- si la liste est réduite à une seule cellule (suivant(l))
 - Terminé : la liste l contient l'adresse de la dernière cellule
 - Renvoyer l
- si la liste est composée de plusieurs cellules
 - renvoyer dernier(suivant(l))

Fonction dernier(l) → Liste

Début

Si esVideListe(suivant(l)) alors

renvoyer l

Sinon

renvoyer dernier(suivant(l))

Finsi

Fin

Insertion dans une liste triée

inserListeTriée : TElement X Liste --> Liste

Schéma récursif

- la liste est vide → insertion en tête de liste
- l'élément à insérer est inférieur ou égal à la donnée de la première cellule → on effectue une insertion en tête de liste
- l'élément à insérer est supérieur à la donnée de la première cellule
→ On effectue l'insertion par rapport à la sous liste.

procédure inserElementListeTriee(elt, l)

P.F elt : TElement (E) l : Liste (E/S)

Debut

Si (estVide(l)) ou (elt <= donnee(l)) alors

InserTete(elt , l)

Sinon

InserElementListeTriee(elt, suivant(l))

Finsi // **Attention traduction en C**

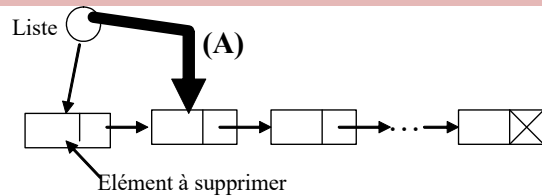
Fin

Schéma itératif

Suppression d'un élément dans une liste

Suppression du premier élément

suppTête : Liste--> Liste



Pré-condition : On suppose que la liste n'est pas vide.

Remarque : pour libérer la mémoire occupée par la première cellule, il faut donc préserver l'adresse de la tête de la liste avant la modification du chaînage

procédure suppTete(l)

P.F I : Liste (E/S)

Début

ll ← I /* Préservation de l'adresse de la première cellule */

I ← suivant(I) /* Chaînage A */

libererMemoire(ll)

Fin

void suppTete(Liste *I){

Liste ll ;

ll = *I ;

*I = suivant(*I) ;

free(ll) ;

}

Liste suppTete(Liste I){

Liste ll ;

ll = I ;

I = suivant(I) ;

free(ll) ;

return I ;

}

Suppression de la liste

suppListe : Liste--> Liste Vide

Version itérative

Procédure suppListe(l)

P.F I : Liste(E/S)

Début

Tantque non estVide(l) faire

suppTete(l)

finTantque

Fin

Version Récursive

procédure suppListe(l)

P.F I : Liste(E/S)

Début

Si non estVide(l) alors

suppTete(l)

suppListe(l)

finSi

Fin

Suppression (suite)**Hypothèse : la liste est non vide**

L'élément à supprimer peut-être déterminé par sa position ou par sa valeur.

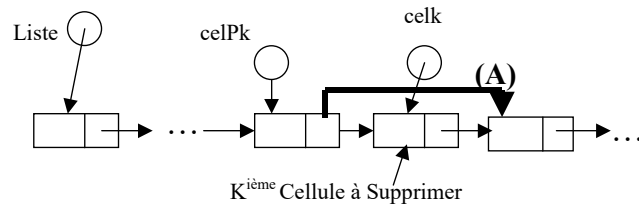
Suppression par position

On souhaite supprimer le $k^{\text{ième}}$ élément d'une liste.

*Schéma itératif**Principe*

Il faut déterminer l'adresse précédente de la cellule qui précède celle que l'on veut supprimer : c'est à dire l'adresse de la $k-1^{\text{ième}}$ cellule.

Si la $k^{\text{ième}}$ cellule existe, en modifié le chaînage suivant comme suit :

**Fonction supPosListe(k, l) → Booléenne**

P.F k : Entier (E) l : Liste (E/S)

Début

Si (k=1 et non estVide(l)) alors

SuppTete(l)

Renvoyer Vrai

Sinon

Suppression ← faux

celPk ← accesPositionListe(pos-1, l)

Si non estVideListe(celPk) alors

celK ← celPk.suivant

Si non videListe(celK) alors

Suppression ← Vrai

celPk.suivant ← celK.suivant

LibérerMémoire(celK)

Finsi

Finsi

Renvoyer suppression

Finsi

Fin

// attention traduction en C

En utilisant la fonction `suppTete`, on obtient la version suivante :

Fonction `suppPosListe(k, l) → Booléenne`

P.F `k : Entier (E)` `l : Liste (E/S)`

Début

Si (`k=1` et non `estVide(l)`) alors

`SuppTete(l)`

 Renvoyer Vrai

Sinon

`Suppression ← faux`

`celPk ← accesPositionListe(pos-1, l)`

 Si non `estVideListe(celPk)` alors

 Si non `estVideListe(suivant(celPk))` alors

`Suppression ← Vrai`

`SuppTete(celPk) //Attention traduction C`

 Finsi

 Finsi

 Renvoyer `suppression`

Finsi

Fin **// attention traduction en C**

Schéma récursif

- si la liste est vide
 → Suppression est impossible
- si la liste est non vide
 ** `k=1` →
 suppression en tête de liste
 `suppTete(l)`
- ** `k>1` ->
 suppression du `k-1` ième élément par rapport à la sous liste
 `suppPosListe(k-1, suivant(l))`

Fonction `suppPosListe(k, l) → Booléenne`

Début

Si `estVide(l)` alors

 Renvoyer Faux

Sinon

 Si `k=1` alors

`SuppTete(l)`

 renvoyer Vrai

 Sinon

 Renvoyer `suppPosListe(k-1, suivant(l))`

 Finsi

Finsi **// attention traduction en C**

Fin

Suppression d'un élément dans une liste

Schéma récursif

- si la liste est vide
 - Suppression est impossible
- si la liste n'est pas vide
 - si l'élément à supprimer est égale à la donnée de la 1^{er} cellule
 - Suppression en tête de liste
 - si non la suppression s'effectue par rapport à la sous liste

Fonction SuppElementListe(elt , l) → booléenne

Début

Si esVideListe(l) alors

Renvoyer Faux

Sinon

Si donnee(l) =elt alors

suppTete(l)

Renvoyer Vrai

Sinon

Renvoyer SuppElementListe(elt, suivant(l))

Finsi

Finsi

Fin

// attention traduction en C

Schéma itératif

Fonction SuppElementListe(elt , l) → booléenne

Début

Si estVide(l) alors

renvoyer Faux

Sinon

Si donnee(l) =elt alors

suppTete(l)

renvoyer Vrai

Sinon

ll ← suivant(l)

suppression ← faux

Tantque (non estVideListe(ll) et donnee(ll)≠elt) faire

ll ← suivant(ll)

Fintq

Si (non estVideListe(ll) et donnee(ll)=elt) alors

suppTete(ll)

suppression ← Vrai

Finsi

Renvoyer Suppression

Finsi

Fin

// attention traduction en C

LISTES AVEC DEUX ADRESSES

Remarque

Pour améliorer l'insertion en queue, nous modifions la structure en rajoutant un pointeur supplémentaire qui indique l'adresse de la dernière cellule.

Déclaration la structure en C

```
typedef struct cellule{
    TElement donnee;
    struct cellule *Suivant;
}* Liste;
```

```
typedef struct {
    [int taille ;           // taille de la liste]
    Liste tete ;           // adresse de la première cellule
    Liste queue ;          // adresse de la dernière cellule
} ListeTQ ;
```

Primitives de base de la structure Liste

Précondition : non vide liste

Donnée : retourne la donnée d'une liste

Donnee : Liste → TElement

Fonction donnee(l) → TElement

P.F l : Liste (E)

Début

renvoyer donnee(tete(l))

FIN

suivant : retourne l'adresse de la cellule suivante

suivant : Liste → Liste

Fonction suivant(l) → Liste

P.F l : Liste (E)

Début

renvoyer suivant(l)

FIN

Primitives de base de la structure ListeTQ

tete : retourne la cellule en tête de liste

tete : ListeTQ $\rightarrow$ Liste

Fonction tete(l) $\rightarrow$ Liste

P.F l : ListeTQ

Début

renvoyer l.tete

FIN

queue : retourne la cellule en queue de liste

queue : ListeTQ $\rightarrow$ Liste

Fonction queue(l) $\rightarrow$ Liste

P.F l : ListeTQ

Début

renvoyer l.queue

FIN

Fonction initLTQ : [] $\rightarrow$ ListeTQ

Fonction initLTQ() $\rightarrow$ ListeTQ

Début

l.tete $\leftarrow$ l.queue $\leftarrow$ NULL

renvoyer l

FIN

Fonction estVideLTQ : ListeTQ $\rightarrow$ Boolean

Fonction estVideLTQ(l) $\rightarrow$ Booléenne

P.F l : ListeTQ (E)

Debut

renvoyer **estVide(tete(l)) && estVide(queue(l))**

Fin

Fonction estPleinLTQ : Liste $\rightarrow$ Boolean

Fonction estPleinLTQ(l) $\rightarrow$ Booléenne

P.F l : ListeTQ (E)

Debut

Renvoyer **faux**

Fin

Insertion en tête

InsertTete : TElement x ListeTQ → ListeTQ

Procédure inserTete(elt, l)

P.F

elt : TElement (E)

l : ListeTQ(E/S)

Debut

cel ← allocMemoire(Cellule)

cel.donnee ← elt

Si estVideLTQ(l) alors

cel.suivant ← NULL

l.tete ← l.queue ← cel

Sinon

cel.suivant ← tete(l)

l.tete ← cel

Fsi

FIN

Fonction inserTete(elt, l) → ListeTQ

P.F

elt : TElement (E)

l : ListeTQ(E)

Debut

cel ← allocMemoire(Cellule)

cel.donnee ← elt

Si estVideLTQ(l) alors

cel.suivant ← NULL

l.tete ← l.queue ← cel

Sinon

cel.suivant ← tete(l)

l.tete ← cel

Fsi

Renvoyer l

FIN

Insertion en queue

InserQueue : TElement x ListeTQ → ListeTQ

Procédure inserQueue(elt, l)

P.F elt : TElement (E) l : ListeTQ (E/S)

Début

```

cel ← allocMemoire(Cellule)
cel.donnee ← elt
cel.suivant ← NULL
Si ( estVideLTQ(l) )
    l.tete ← l.queue ← cel
else
    //Insertion en Queue d'une Liste
    l.queue.suivant ← cel
    l.queue ← cel
fsi

```

FIN

Fonction inserQueue(elt, l) → ListeTQ

P.F elt : TElement (E) l : ListeTQ (E)

Début

```

cel ← allocMemoire(Cellule)
cel.donnee ← elt
cel.suivant ← NULL
Si ( estVideLTQ(l) )
    l.tete ← l.queue ← cel
else
    //Insertion en Queue d'une Liste
    l.queue.suivant ← cel
    l.queue ← cel
fsi
renvoyer l

```

FIN

Suppression en tête

Précondition : non est vide listeTQ

suppTete : ListeTQ → ListeTQ

Procédure suppTête(l)

P.F l : ListeTQ (E/S)

Début

```

ll ← tete(l)           // On mémorise la tête
l.tete ← suivant( tete(l) ) // passage au suivant
si estVideL(tete(l)) alors
    l.queue ← NULL
fsi
libMemoire(ll) // libération de la mémoire occupée par l'ancienne tête

```

FIN

Fonction suppTête(l) → Liste

P.F l : ListeTQ (E)

Début

```

ll ← tete           // On mémorise la tête
l.tete ← suivant( tete(l) ) // passage au suivant
si estVideL(tete(l)) alors
    l.queue ← NULL
fsi
libMemoire(ll) // libération de la mémoire occupée par l'ancienne tête
renvoyer l

```

FIN